

CSU - RAMS

Model Overview and Installation Guide

This document provides an overview of RAMS, contains detailed notes on how to setup the RAMS software and all the pre-cursor third party software needed to run the model, contains examples of Linux .bash_profile, describes how to pre-process gridded atmospheric data for the model, describes how to run the model and post-process the output, describes how to perform the test simulation, describes the code formatting for developers, and provides some tips on troubleshooting.

Updated by:

**Stephen Saleeby
Department of Atmospheric Science
Colorado State University**

Last updated: August 30, 2026

GENERAL SOFTWARE AND RAMS INSTALLATION INSTRUCTIONS
#####

RAMS IS DESIGNED TO RUN ON A LINUX x86_64 PLATFORM. IF YOU HAVE A DIFFERENT PLATFORM, CERTAIN SECTIONS OF THE CODE WILL NEED TO BE CUSTOMIZED BEFORE THE MODEL CAN BE COMPILED SUCCESSFULLY AND USED.

RAMS IS COMPOSED OF C AND FORTRAN-90 FORMAT CODE. YOU MUST HAVE C AND FORTRAN COMPILERS INSTALLED ON YOUR SYSTEM BEFORE PROCEEDING. "GCC" IS THE MOST COMMONLY USED C COMPILER WITH RAMS BUT HAS ALSO WORKED WITH "CC", "MPICC", AND "ICC" ON SOME SUPERCOMPUTERS. WE HAVE SUCCESSFULLY TESTED RAMS WITH THE FORTRAN COMPILERS "PGF90", "IFORT", AND "GFORTRAN". IT HAS COMPILED WITH "MPIF90" ON SOME SUPERCOMPUTERS. OTHER COMPILERS MAY NOT WORK WITHOUT CODE MODIFICATIONS. NOTE THAT RAMS COMPILED WITH "GFORTRAN" RUNS MUCH SLOWER THAN OTHER COMPILERS AND THE CODE CAN BE PROBLEMATIC AT RUNTIME. GFORTRAN SEEMS TO FLAG ERRORS THAT ARE OFTEN NOT REALLY ERRORS SUCH AS "ARRAY BOUNDS" THAT ARE NOT ACTUALLY A PROBLEM.

Outline:

Directory description and overview

- 1: include.mk - Compiling control file usage
- 2: bin.rams - Compile and test RAMS model
- 3: bin.dp.grib1 - Preparing Grib-1 gridded datasets for case study
- 4: bin.dp.grib2 - Preparing Grib-2 gridded datasets for case study
- 5: bin.block - Custom development of lat/lon block surface files
- 6: docs - RAMS documentation
- 7: src - Source code
- 8: etc - Extra required input custom data files

PREREQUISITES: Software that needs to be installed for RAMS

APPENDIX A: Troubleshooting RAMS and common issues

APPENDIX B: Setting up compilers, HDF5, NETCDF, MPI, SZIP, ZLIB

APPENDIX C: Tech and coding specifications for RAMS

APPENDIX D: Notes and examples on using wgrib and wgrib2

APPENDIX E: Useful Linux commands

APPENDIX F: Julian day Listing

#####

Directory description and overview

#####2

1. To clone RAMS from GitHub:

```
cd ~/whatever/directory/you/use
```

```
git clone https://github.com/RAMSmodel/RAMS.git
```

```
cd RAMS
```

2. Enter the main RAMS directory. this top-level directory path will be the "RAMS_ROOT" environmental variable used in subsequent steps in the "include.mk" compile control file. Modify the "include.mk" compile control file with correct paths to the RAMS source code, RAMS version number, linking libraries, HDF5, NETCDF C & F, MPICH, Fortran compiler and flags, C compiler and flags, and MPI flag section. Get to know your "include.mk" file! Changes will need to be made for different computers in order to compile the model. For an initial serial compilation (running on only 1 processor), comment out the "PAR_" flags at the bottom of "include.mk".

3. The step above noted that you need to have the following on your Linux system: (1) Fortran and C compilers, (2) MPICH for parallel processing, (3) HDF5 for doing input/output data, (4) SZIP and/or ZLIB libraries for data compression in HDF5, (5) ZFP software for optional LITE file data compression, and (6) NETCDF for the RRTM radiation scheme. See the section on "Prerequisites" below regarding these software packages that need to be installed on Linux and pointed to in your "include.mk" file for compilation.

4. Once your "include.mk" file is correct for your computer system, enter "bin.rams" directory and type "make clean" to clean up an old compilation, and then type "make" to compile the model. Any errors that show up are likely due to issues with settings in "include.mk". You will need to fix any such discrepancies. Follow the errors messages. They often have to do with linking libraries that you need to point to in "include.mk" or have in your environment path in Linux.

5. Once the model is compiled, it will produce an executable like "rams-6.3.05".

6. Once you have compiled the model you can test it with the example simulation script within "bin.rams". The "run_testsupercell.sc" script will execute a test simulation to see if your compilation worked. This script makes use of the "RAMSIN" name-list file that the user modifies for the type of simulation that is desired. The sample one is "RAMSIN.supercelltest". See the RAMSIN name-list user guide in the "docs" directory on the settings in these files. For a simple test, run on 1 processor, you could execute RAMS as something like "rams-6.3.05 -f RAMSIN.supercelltest". You would have to make sure that output data directories given in RAMSIN.supercelltest are already created. Try this out and get to know RAMS. More details are in the section on "bin.rams", below.

Below is a description of the files and directories you see in this top-level location. We will step through the files and directories in an order that will allow you to compile and test the various components of the model as you read through this document.

#####

1. include.mk – Compiling control file usage

#####

This FILE is the compile control file.

This is where the user sets environmental and directory paths for RAMS source code, HDF5, MPI, and FORTRAN and C compilers as well as the compiler flags and required libraries. It is the only file that needs to be modified prior to compiling within the various RAMS top level “bin” directories. This file requires correct setup before any compilation will work.

Edit the include.mk file. Several things in this file need to be changed/set:

- a. Change the RAMS_ROOT variable to point to the top-level directory and change the RAMS_VERSION number if necessary.
- b. HDF5 directory location – HDF5_ROOT. Set HDF5_LIBS to the correct library paths of your HDF5 installation. Set HDF5_INCS to the directory where the C include file ‘hdf5.h’ exists. Leave HDF5_DEFS alone for now. If you are running on a supercomputer, use the native computer versions installed as modules. Commands on supercomputers such as “module avail” or “module list” can provide info on what software is available and how to load it.
- c. MPI directory location – MPI_ROOT. Set PAR_INCS to the directory where the MPI include file exists. Set PAR_LIBS to the correct library paths of your MPI installation. Set PAR_DEFS = -DRAMS_MPI. If you are running on a supercomputer, it is best to use their MPICH modules. See their instructions on how to load modules.
- d. NETCDF directory location – NETCDF_FORTRAN_ROOT and NETCDF_C_ROOT need to point to these install packages.
- e. Machine-dependent options:
CMACH - name of machine/compiler type. Used in source files to conditionally compile code sections. Our include file and code currently only contains ‘PC_LINUX1’ option for Linux framework. If you have a different platform, search for ‘PC_LINUX1’ to see where you need to make modifications. These are related to specific command line argument collection, clock timing methods, and use or non-use of C subroutine underscores. You may need to examine these files and modify code for your system since PC_LINUX1 is our currently tested platform.
- e. Fortran and C compiler flags:
There are default Fortran compiler flags already included for PGF90, IFORT, and GFORTRAN; uncomment only one set of these and insert the proper path to compiler. The Fortran flags need to either reflect the single or double precision variable option. There are default settings for both in this file. These settings are initially set to single precision. Note that double precision will reduce runtime, use more memory, and increase the output file size substantially.
F_COMP - name of Fortran compiler used to compile all the software packages noted above.
F_OPTS1,2,3 - command line options for the Fortran compiler.
C_COMP - name of C compiler.

C_OPTS - command line options for the C compiler.

LOADER_OPTS - options for linking.

LIBS - any other libraries required for linking.

ARCHIVE - archive command. Typically, 'ar rs'.

F_OPTS1, F_OPTS2, & F_OPTS3 can be the same or different. We have found that we need to compile some Fortran files with different optimization levels; thus, we typically have these two flags differ in their optimization level only. In the RAMS MAKEFILE you can see which files are compiled differently from others.

C_OPTS may contain an optional flag "--DRAMS_DOUBLE_PREC" to turn on double precision. We use single precision by default. There are notes within "include.mk" on using this.

C_OPTS may contain an optional flag "--DENABLE_PARALLEL_COMPRESSION" to turn on HDF5 data compression as model output is written. This is now used by default but sometimes does not work. If it fails, it always does so the very first time a RAMS output file is written at initialization. Try using this by default since it saves the steps of have to compress HDF5 RAMS output data files after a simulation is finished. If you need to compress data after a simulation is finished, you can use the HDF5 "h5repack" tool supplied in the HDF5 package you installed on Linux.

Once you verify that you have: (1) your Fortran and C compilers, HDF5, NETCDF-C, NETCDF-FORTRAN, MPICH, SZIP, ZLIB installed, (2) the correct settings of flags in "include.mk", (3) variables "PATH" and "LD_LIBRARY_PATH" set in your environment variables file (ex: .bash_profile or .profile), then proceed to try and compile RAMS in the "bin.rams" directory as discussed in the following section.

#####

2. bin.rams – Compile and test RAMS model

#####

This DIRECTORY is the primary RAMS model compile directory.

Enter the bin.rams directory. Perform a fresh compile by typing "make clean" and then "make". Here you will create an executable named something like "rams-6.3.05". If you get a compile error, please see section on troubleshooting or follow the error messages. They often have to do with library linking.

The executable is controlled by the "RAMSIN" namelist control file which is used to specify your simulation setup and choices of parameterizations. A detailed description of each namelist variable is given in the docs file RAMS-Namelist.pdf. Simple sequential (single-processor) RAMS model execution is done with the syntax example "rams-6.3.05 -f RAMSIN". You can change the name of the RAMSIN namelist as long as you specify the namelist being accessed. Parallel execution varies with MPI software. A sample execution using MPICH3 and machines file "machs" would be something like:

"mpiexe -machinefile machs -np 12 rams-6.3.05 -f RAMSIN".

In general, there are 2 main types of simulations that can be run with RAMS: (1) idealized, and (2) case studies. Idealized simulations do not require the geographic surface data in “sfctypehdf5” or any data pre-processing of gridded reanalysis products since they are typically initialized from a sounding and user-specified surface characteristics in the RAMSIN namelist and do not use boundary nudging. Variable initialization case study simulations that are tied to specific and real geographic locations require that you point to the location of “sfctypehdf5” in RAMSIN. Case studies also require that you provide gridded reanalysis type data in grib-1 or grib-2 format that is formatted into “dp” files via the data preprocessors in “bin.dp.grib1” or “bin.dp.grib2”, respectively. The “dp” files created in the bin.dp.grib directories is pointed to in the RAMSIN in order to generate the varfiles for variable initialization. The RAMSIN namelist guide provides all the details of the settings.

There is a test simulation here:

1. Idealized supercell thunderstorm test on small domain using a sounding for horizontally homogeneous initialization (Leah Grant et al. 2014,2015, JAS). The script "run_supercelltest.sc" can be used to execute this test. Standard runtime output (not model analysis files) is sent to the screen for viewing progress of the simulation. It is a very fast running simulation used to see that your installation is working for a basic simulation. The script uses the RAMSIN namelist file "RAMSIN.supercelltest". The output from this simulation is sent to directory "testsupercell.output" which is created automatically in the execution script.

2. Note that the test script, run_supercelltest.sc, is set up to run in sequential mode and does not use parallel processing. It will run faster if you enable parallel processing. Please examine these scripts to understand their usage and how to turn on/off the parallel processing. If you turn on parallel processing, via the script flags "runtype" and "n" (for number of processors), the script will attempt to create a machines files (or "machs" file) based on the hostname of the current machine. If your current machine is not multi-core or does not contain the minimum number of cores specified by the script variable "n", then the default parallel test simulation setup will not run correctly without customization of a "machs" file that is appropriate for your computer and MPI software. This is why the default scripts for executing these test simulations are setup to run sequentially and not in parallel. A sample “machs” file named “machs.master” is provided in “bin.rams” and contains several commented-out sections that could be used in a similar manner on a local cluster if you have on. Each cluster is different, so the “machs” file would need to be customized for your machine. Such a “machs” file is not needed if running on a supercomputer that uses a “PBS” queueing system. You would need to write your own script to run their “PBS” system with a “qsub” command or something similar.

#####

3. bin.dp.grib1 - Preparing Grib-1 gridded datasets for case study

#####

This DIRECTORY is the compile directory for the RAMS GRIB-1 pre-processor.

Enter the bin.dp.grib1 directory. Perform a fresh compile by executing the "make.sc" bash script. Here you will create an executable named something like “dgrib-6.3.00”.

The make.sc script must first compile “rams_wgrib.c” and tries to use the default gcc compiler. If gcc cannot compile this “.c” file, then you will need to modify make.sc and try a C compiler on your computer that works.

What the de-gribber does

This de-gribbing software is used to extract and convert Grib-1 formatted gridded atmospheric pressure level data into RALPH-2 ASCII-format dataprep or "dp" files that RAMS uses for making variable initialization files or "varfiles" used for real case study initialization and nudging. The "dp" files have header information at the top followed by the data that is ordered in the file according to the order of the variables listed in the standard output that is sent to the screen when running the de-gribbing executable.

These “dp” data are needed for case studies in which heterogeneous initialization and lateral boundary or central data nudging are necessary. These are not needed for idealized type simulations that use homogeneous initialization from soundings and no nudging. More specific information on preparing gridded atmospheric data, observed rawinsonde data, or observed surface data can be found in the docs file RAMS-DataPrep.pdf. Note that for real case study simulations, only processing of gridded datasets is required; rawinsonde and surface observations are not necessary, but they can be input as a customization used by the model in the observational data assimilation (ODA) section noted in the RAMSIN namelist.

What the de-gribber extracts

Atmospheric fields:

The degribber code always tries to extract the atmospheric fields of u-wind, v-wind, relative humidity or specific humidity, temperature, and geo-potential height. It assumes winds are earth relative. There are some customized exceptions that are handled on a case-by-case basis and are mentioned below. If a dataset does not contain these default conventions, then customization is required in the dataprep source code in dprep/dgrib1_main.f90 and in lib/griber_grb1.c for Grib-1 format.

Surface fields.

The code also tries to extract soil moisture, soil temperature, snow depth, and snow water equivalent when provided.

Executing the de-gribber

Run as "dgrib-<version> -t # -d YYYYMMDDHH -h fcsthr -f filename"

1. -t specifies the data set being used (number as above)
2. -d is the date and time
3. -h is the forecast hour (usually zero for reanalysis)
4. -f is the path to the file to convert

Where the -t argument could be “1” for NCEP-reanalysis.

Where the -d argument could be “2007021100” for 00Z Feb 2, 2007.

Where the -h argument could be “0” for forecast hr 0 (almost always 0).
Where the -f argument could be “/home/smsaleeb/NARR_20070211.grb”.

Please see the separate dataprep documentation “RAMS-DataPrep.pdf” for details on what data can currently be processed and how to process it. Please take care when using this code because every dataset is a bit different and this code in the files dprep/dgrib1_main.f90 and lib/griber_grb1.c are customized to add or modify processing of GRIB-1 gridded data.

4. bin.dp.grib2 - Preparing Grib-2 gridded datasets for case study

This DIRECTORY is the same as bin.dp.grib1 but for working with gridded data in GRIB-2 format.

Enter the bin.dp.grib2 directory. Perform a fresh compile by executing "make clean" and then "make".

For this de-gribbing code to run:

This software requires that the wgrib2 software be correctly compiled on your machine. Here we have provided a precompiled executable "wgrib2" and the wgrib2 software source code wgrib2.v2.0.3.tar.gz. If the precompiled executable does not work for you, then you will need to try to compile the code in wgrib2.v2.0.3.tar on your computer and place the generated executable "wgrib2" within this current directory in place of the provided precompiled executable. To compile wgrib2 from source, unzip and untar the wgrib2 source code provided; enter the main directory. Read the INSTALL file and “makefile” to be a bit familiar with them. You might need to make a few changes. However, you can first try typing “make” to see if the installation settings work as is. If so, then you will find a “wgrib2” executable within the “wgrib2” directory. You can copy this executable to the bin.dp.grib2 directory and try out the example tests as discussed below.

This code for Grib-2 format accomplishes the same task that the section above for Grib-1 format. It is just that it uses the Grib-2 de-gribber uses the code dprep/dgrib2_main.f90 and lib/griber_grb2.c which are similar but account for some differences between Grib-1 and Grib-2. The big difference with Grib-2 is that the third-party “wgrib2” software is a bit more complex to install than “wgrib1”.

Variable names in Grib-2 format files may be different that the Grib-1 names. This is mostly accounted for in the code, but some exceptions occur.

It is advised to examine some of the differences in dgrib1_main.f90 and dgrib2_main.f90. This will highlight the differences in variable names in the documentation near the beginning of these files as well as what datasets currently work with the degribber. Additional dataset options can be added as needed.

5. bin.block - Custom development of lat/lon block surface files

This DIRECTORY is the compile directory for creating CUSTOMIZED surface data latitude-longitude block files for RAMS. (Not used by typical RAMS user.)

Enter directory bin.block. Perform a fresh compile by typing "make clean" and then "make".

All of the static surface characteristics data in "sfctypehdf5" (ie. vegetation, soil type, topography, SST) are in block file format, meaning that multiple files are used to cover certain latitude longitude blocks of the earth rather than having a single very large file that covers the entire earth. The RAMS model is programmed to read this specific file format for surface characteristics. The example setup in this directory is for creating SST files from observational data. To get info on how to execute, just run the executable after compilation, and usage information will be displayed to the screen. (This requires in-depth model knowledge and customization for any usage.)

To test BLOCK file making, you can use sample MODIS-4km and REYNOLDS-1-deg SST data in "example_modis_sst" and "example_reynolds_sst" that has been pre-prepared in a format that the blockfile making code can ingest (These are not provided in the standard model release but are available upon request). Just modify and run the script run_test_sstfiles.sc. This will output new SST block files that are in a format that RAMS can ingest for making runtime surface files in a MAKESFC run. The scripts will automatically compare the newly generated block files to some pre-existing ones that are in the example directories.

To run the block file executable:

executable input_file datatype output_directory prefix

Whereby:

input_file - name of input data file
datatype - 1=modis9kmsst, 2=1degReynoldssst
output_directory - directory where header file will be written
prefix - filename prefix for header and data files

Example execution with sample file names and such:

mk-blkfiles-6.1.0 131231_ 1 output S

See SST example in directory example_modis_sst.

File 131231_ is ASCII lat,lon,SST(C) MODIS data.

Files S131231_90S000E.h5 S131231_90S180W.h5 are output for RAMS SST.

File SHEADER is the header file.

Note that this process is CUSTOMIZABLE and not UNIVERSAL. Any variation from the data examples included must be accompanied by modification to the code in the "block" source code directory within file "mk_main.f90".

#####

6. docs – RAMS documentation

#####

This DIRECTORY contains the files of RAMS documentation.

There is ample documentation regarding many aspects of the RAMS model. Several key documents to note in order to run the analyze model results are as follows.

1. RAMS-Namelist – This describes all of the key parameters that need to be set in the RAMSIN namelist file in order to start a RAMS simulation.
2. RAMS-VariableList – This contains a list of all possible RAMS output variables that could be contained in the output analysis files. It contains the name, description, and variable units.
3. RAMS-Updates – Provides a detailed log of changes from one version of the model to the next.
4. RAMS-DataPrep – Provides a detailed description of the formatting of the dataprep or “dp” files that are generated from gridded Grib format datasets. These “dp” files are used by RAMS to create the variable initialization and nudging files or “varfiles”. The varfiles are then used to initialize and/or nudge the model in case study simulations.
5. RAMS-TechnicalManual – Provides a summary of the parameterizations and mathematical techniques used in RAMS.
6. RAMS-Leaf2-to-Leaf3 – This provides some information on the changes in the LEAF land-surface model from versions 2 to 3.
7. RAMS-HUCM-Bin-Micro – This provides information on how to run RAMS with the Hebrew University Cloud Model with Spectral Bin Microphysics. It also provides information on the output variable names.

#####

7. src – Source code

#####

This DIRECTORY contains the directory tree to RAMS source code.

#####

8. etc – Extra required input custom data files

#####

This DIRECTORY contains files that the model accesses for information related to particular subroutines.

Note that the location of this directory and files will need to be input into the RAMSIN namelist at runtime.

#####

PREREQUISITES: Software that needs to be installed for RAMS

#####

This section discusses the necessary HDF5, NETCDF-Fortran, NETCDF-C, MPICH, ZFP compression, SZIP, ZLIB software. RAMS needs the executables and/or libraries and include files associated with these packages. HDF5 needs the SZIP and/or ZLIB libraries. Your user environment specified in Linux files such as “.bash_profile”, likely needs to point to these “bin”, “lib”, and “include” directories in the “PATH=” environment variable. The “LD_LIBRARY_PATH=” likely also needs to include the paths to the libraries for each of the software packages. A lot of this depends on how your Linux system and user environment are set up. If you get compile errors with RAMS, pay attention to the error messages, as they often let you know if certain executables, include files, and libraries are not accessible. Errors usually relate to the linking in “include.mk” or your system environment variables. Once you have a compiled RAMS executable such as “rams-6.3.05” you can run “ld rams-6.3.05” to see which libraries are linked to make sure you are using the expected versions if you have multiple versions on your Linux system. Normally a full RAMS compile with take a couple of minutes.

First, we start here if your computer system has the PGI (pgf90) or INTEL (ifort) Fortran compilers and a C compiler (gcc, icc). RAMS may not work with GFORTRAN (f90).

RAMS will not compile without these third-party software packages. Attention to this must come first before proceeding. If this software is already on your computer system or supercomputer that you are using, then you do not need to install these. Just make sure your Linux system includes the “PATH” and/or “LD_LIBRARY_PATH” to these software packages. and that your “include.mk” compile file points to the necessary software.

The included software is needed by RAMS for parallel input/output (HDF5) and parallel processing (MPICH). RAMS input/output is done with parallel-HDF5 for use on massively parallel systems. The included third-party software are versions that we use frequently and are well tested with RAMS. You can install other versions if you wish. However, you must have some version of parallel-enabled HDF5 installed on your computer. Parallel-enabled HDF5 requires Message Passing Interface (MPI) software installed on your computer. Install these packages somewhere on your computer system and then make sure your “include.mk” compile control file points to the paths for this software. Also make sure your Linux “.bash_profile” type environment file sources the locations of HDF5, MPICH, SZIP, and ZLIB executables/libraries in the “PATH” and “LD_LIBRARY_PATH” environment variables.

RAMS and third-party software need C libraries, so “glibc” and a C compiler need to be available on your computer system and they usually are by default on Linux systems. As you move RAMS between systems or have Linux OS upgrades, you might need to recompile third-party software and RAMS due to changes in “glibc” and other system libraries.

See section on “include.mk” for more details on use of these necessary software packages.
See APPENDIX for setting up this prerequisite software and suggested versions.

#####

APPENDIX A: Troubleshooting RAMS and Common Issues

#####

1. TO PROCESS OLD ANALYSIS FILES WITH NEWER REVU:

We used array swap starting in RAMS 6.1.12. To process old data with new REVU code, look in the file "lib/hdf5_utils.f90" for notes by "Saleeby" about a few lines of code to change. Do not forget to change this back after working with old data.

2. TO PROCESS RADAR PARAMETERS SUCH AS D0, LOGNW, ETC. FROM REVU WITH OLD DATA:

Some older simulations do not contain GNU (hydrometeor shape parameter) in the header files, so we need to temporarily hardcode the value of GNU used in the older simulations we want to examine. This can be done in subroutine "hydrogamma" in the file "revu/rcomp.f90". See notes by "Saleeby".

3. CANNOT FIND COMPILER, HDF, OR MPI

Most compile problems and error messages tend to be related to improper installation of FORTRAN COMPILER, HDF5 or MPI. Please examine the error messages to see what is missing. Often, the wrong environmental path is given in the "include.mk" compiling control file.

4. COMPILE TIME LINKING ERRORS:

A frequent error message occurs at the very end of compilation related to the linking of libraries. This is usually related to locations of MPI, HDF5, or LINUX system libraries. Also, these software packages often need certain LINUX system libraries that are not initially apparent. Please make sure your computer system has the libraries that appear in any error message, and then add linking library files as needed.

5. RANDOM SEGMENTATION FAULTS:

Sometimes running the model will generate random segmentation faults that cannot be tied to a particular line of code for missed memory allocation. Rather, the computer stack memory may be overflowing. To avoid this, you can increase your Linux system stack size or set it to unlimited. There should be no real problem with this. The command to do this is "ulimit -s unlimited".

6. RANDOM CRASHES DUE TO MAX VARIABLE HARDCODED LIMITS

When you first run the model, note that most memory is dynamically allocated. However, since some variables must be declared before the model has the chance to read the RAMSIN namelist to obtain model dimensions and such, some variables must be given "max" dimensions. The source file "memory/grid_dims.f90" contains a list of "max" dimensions. You should adjust these as needed for your simulations. The dimensions are currently set to fairly large values, but as modeling capabilities improve, sometimes some of the values of max dimensions need to be increased.

7. RUNTIME "TOO MANY CCN" TYPE ERROR:

This error is usually indicative of numerical problems near topography. Try reducing your timestep and/or using RAMSIN variable IHORGRAD=2 to deal with diffusion near steep

topography. You can also try smoothing your topography to prevent adjacent grid cells from having too large of a jump in elevation, which creates the error when using IHORGRAD=1.

8. ISSUES THAT OCCUR WHEN RUNNING WITH FINE VERTICAL SPACING OVER TERRAIN:

First try smoothing terrain using RAMSIN namelist settings for topography representation and topography wavelength. Also try IHORGRAD=2 which allows for small DZ.

9. SMALL NUMBER ARITHMETIC:

Some compilers (IFORT) may take arithmetic of $(0.0 + 1.e-12)$ as equal to 0.0 if the exponential is quite small. If this sum is in a denominator, this will result in NaN. There are many places in RAMS that do this to prevent division by zero and some, but not all, of them have been replaced with something like $[\max(0.0, 1.e-12) = 1.e-12]$.

10. ARRAY BOUNDS ERRORS:

Note that NaNs can occur due to array bounds errors. So, if a NaN occurs, turn on bounds checking in the compiler flags to isolate the error location and fix the bounds issue.

11. UNINITIALIZED VARIABLES:

Note that NaNs can occur from uninitialized variables. Some compilers just assign a value of zero and others assign NaN. Turn on compiler checking flags for finding uninitialized variables.

12. Error message: UNDEFINED REFERENCE TO...

One cause of this compiler message could be trying to declare a function within a subroutine when both are in the same module. Only declare the function if they are NOT in the same module.

13. Error message: NEGATIVE CONDENSATE MICRO

This error indicates a problem exists but not the source. This can be caused by CFL errors, cumulus parameterization, diffusion, or advection issues. To check further, turn on ICHECKMIC in the RAMSIN namelist. See if other error messages appear. Most times, using a shorter timestep solves the problem.

14. MPI ERRORS:

MPI routines hardcode the variables “nwords” in order to set the buffer size for memory before passing data to the nodes in parallel. When adding variables to the put/get list, “nwords” has to be incremented. If the buffer is not big enough, the model will crash and give errors such as “error stack”, or “size of data to pack is larger than remaining space on buffer”.

15. INITIALIZATION IN PARALLEL:

Some initialization needs to be done on a single node and then distributed to all the nodes. As an example, the random bubble generator (IBUBBLE=3 in RAMSIN) will create different perturbations if the sub-domain sizes are different for different numbers of nodes being used for parallel processing. We can get around this by letting a single node do the randomizer at model start and then broadcasting the bubble field to the nodes.

16. NON-INCREMENTING DO LOOPS

Some compilers do not like non-incrementing DO loops such as [DO N=K1,K2] where K1=K2. In some cases, the loop does not get run, so variables inside the loop are not solved or set. This could lead to NaNs and model crashes.

17. COMPILING HDF5:

In compiling HDF5, the variable H5_CFLAGS may contain the flag "-ansi". This argument tells gcc to use the ISO C90 standard which does not accept C++ style comments which HDF5 seems to use sometimes. So, it recognizes the comment style but will not accept it. The fix is to replace "-ansi" with "-std=c99". This may be contained in "config/gnu-flags" in variable H5_CFLAGS= "\$H5_CFLAGS \$arch -ansi -pedantic". After switching out the "-ansi" flag, rerun "configure" and check inside the Makefile and verify that H5_CFLAGS has "-std=c99" instead of "-ansi". Then try running "make" again.

```
#####  
APPENDIX B: Setting up HDF5, NETCDF, MPICH, SZIP, ZLIB  
#####
```

New Software Setup For RAMS on Workstations (August 2026)

First figure out what Fortran and C compilers are already installed on your system. You will need these for all compilations below. We suggest using INTEL compilers, but PGF90 works well too.

As you do installs and want these custom software versions to take precedence over possible default Linux versions, then make sure these are specified in your "\$PATH" and "\$LD_LIBRARY_PATH" environmental variables before the default ones. Linux looks for the first one it can find.

Run "which" as you do the installs to make sure the correct ones are taking precedence:
which gcc, ifort, mpicc, mpif90, mpifort, nc-config, nf-config, etc...

Then do the following for a bit more info. Can do for "mpif90", "mpicc", etc:
mpif90 -show, mpif90 --version

TO START OUT NEW INSTALLS:

Our installed INTEL compiler is from 2024. Do the following before all the compiles below:
source /opt/intel/oneapi/setvars.sh

Note that newer "IFX" and "ICX" Intel compilers are a problem because they demand every variable is initialized before use. Perhaps there is a way around this, but I am unaware of a solution currently. IFORT will fill uninitialized variable memory with zeros, which is what we have made use of up to now. So, we are using IFORT and GCC for all compiles below.

szip-2.1:

```
./configure --prefix=/home/smsaleeb/software/szip-2.1
make
make check
make install
```

zlib-1.2.5:

```
./configure --prefix=/home/smsaleeb/software/zlib-1.2.5
make
make install
```

MPICH: (start with specifying which INTEL compiler version to use to compile MPICH)

Do NOT put CC, FC, F77 command/path in quotes! But put CPPFLAGS, LDFLAGS in quotes.

Run the configure script making sure your gcc and ifort are the ones you want, otherwise specify the full path in CC, FC, F77. Do NOT put CC, FC, F77 command/path in quotes!

```
./configure --prefix=/home/smsaleeb/software/mpich-4.3.2 \
CC=gcc FC=ifort F77=ifort --enable-fortran=all --enable-shared --enable-static
```

Run make in parallel and install. Do below where “\$(nproc)” is a number of processors:

```
make -j$(nproc)
make install
```

HDF5 (1.12.3 or 1.14.3 with C and Fortran enabled):

Do NOT put CC, FC, F77 command/path in quotes! But put CPPFLAGS, LDFLAGS in quotes.

Run the configure script

```
./configure --prefix=/home/smsaleeb/software/hdf5-1.14.3 \
CC=/home/smsaleeb/software/mpich-4.3.2/bin/mpicc \
FC=/home/smsaleeb/software/mpich-4.3.2/bin/mpif90 \
--enable-parallel --enable-fortran --enable-shared --enable-static
```

Run make in parallel and install

```
make -j$(nproc)
make install
```

NETCDF-C-4.10.1:

Trying to do netcdf-4-c install after MPICH, HDF5.

Do NOT put CC, FC, F77 command/path in quotes! But put CPPFLAGS, LDFLAGS in quotes.

```

../configure --prefix=/home/smsaleeb/software/netcdf-c-4.10.1 \
CC=/home/smsaleeb/software/mpich-4.3.2/bin/mpicc \
CPPFLAGS="-I/home/smsaleeb/software/hdf5-1.14.3/include" \
LDFLAGS="-L/home/smsaleeb/software/hdf5-1.14.3/lib" \
--enable-netcdf-4 --disable-dap --enable-shared --enable-static

```

```

make -j$(nproc)
make check (optional, takes a while to run)
make install

```

```

# Check the install using the following commands
which nc-config, nc-config --version, nc-config --prefix

```

NETCDF-FORTRAN-4.6.4:

Trying to do netcdf-4-fortran install after MPICH, HDF5, NETCDF-C.

Do NOT put CC, FC, F77 command/path in quotes! But put CPPFLAGS, LDFLAGS in quotes.

```

../configure --prefix=/home/smsaleeb/software/netcdf-fortran-4.6.4 \
CC=/home/smsaleeb/software/mpich-4.3.2/bin/mpicc \
FC=/home/smsaleeb/software/mpich-4.3.2/bin/mpifort \
F77=/home/smsaleeb/software/mpich-4.3.2/bin/mpifort \
CPPFLAGS="-I/home/smsaleeb/software/netcdf-c-4.10.1/include" \
LDFLAGS="-L/home/smsaleeb/software/netcdf-c-4.10.1/lib" \
--enable-shared --enable-static

```

```

make -j$(nproc)
make check (optional, takes a while to run)
make install

```

```

# Check the install
which nf-config, nf-config --version, nf-config --prefix

```

#####

APPENDIX C: Tech and Coding specifications for RAMS

#####

Notes on a few specifics if you are contributing code to RAMS. We are starting to force some general coding practice to make doing code searches easier.

Note: all module, subroutine, and function names should be lower case and all calls or usage of them should be lower case. This makes for ease of direct ‘grep’ matching and script searching when examining code usage.

1. Head every module, subroutine, interface, and function with letter case-specific opener. This should begin at column1 and include the spaces and parentheses.

for subroutine example: 'subroutine <routinename> ()'

for module example: 'module <modname>'

for interface example: 'interface'

for function example: 'integer function <funcname>'

2. Use 'implicit none' for every program, subroutine, module, and function and place in first column. The does not have to be added for 'entry' statements within subroutines.

3. End every program, module, subroutine, interface, and function with letter case specific closure:

'End Program main'

'End Subroutine <namehere>'

'End Module <namehere>'

'End Interface'

'End Function <namehere>'

4. Module 'use' statements should be in column 1 before 'implicit none'.

5. For calling subroutines, use 'CALL' in all capital letters, place a space after the name of the subroutine to call, and always use () after the name of the subroutine to call.
(ex. 'CALL cldnuc ()')

6. When using a function in a routine, make sure to declare the function as an 'external' variable and make the function call name all lower case and put parentheses right after the name.
(ex. 'real, external :: rslf') (ie. 'rslf()')

7. For compiling pre-processor code that uses "#ifdef" statements, the fortran files need to end with .F90 rather than .f90.

8. For using the C interface in fortran code to read data from HDF5 files, you must get the C trailing character for fortran to be happy. Example from Quickbeam read of dimensions: [must include the //char(0)].

Call rh5d_open (fileid, 't_coords'//char(0), dsetid, hdferr)

Call rh5a_read_anyscalar (dsetid, 'units'//char(0), stime, RHDF5_TYPE_STRING, hdferr)